

Positional Encodings :

attention is position-blind

∴ position matters, we have to inject position somehow

Old :

★ Absolute positional encoding :

add a fixed vector to each token's embedd & added to beginning.

ISSUE :

it struggles to generalise to sequence length longer than what it's trained

★ Relative positional encoding

: encodes that I am 'n' positions away from token I'm attending to

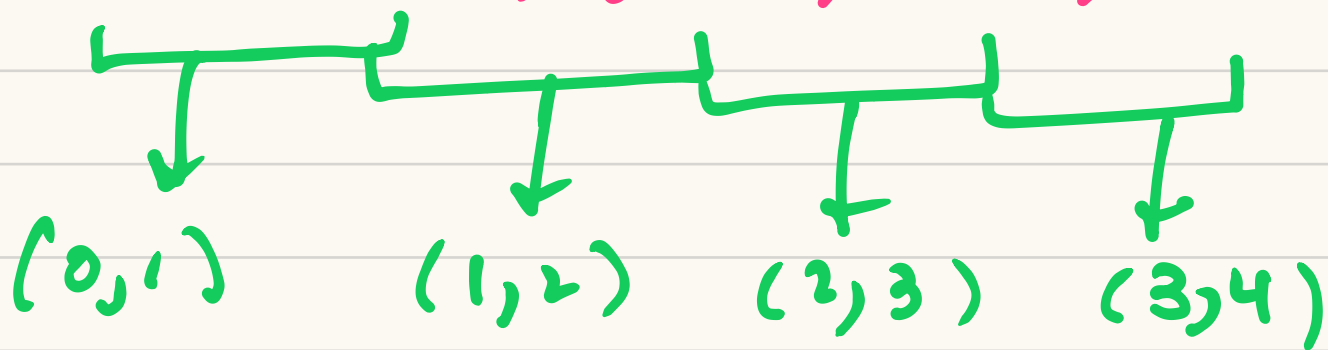
RoPE:

ENCODE POSITION BY ROTATING

Q & K vectors in 2D subspace

Q Vector of dimension d_K

(dim0, dim1, dim2, dim3, dim4, ...)



for each token @ position m
rotate each proportional to m

i.e. angle : $m\theta$ (diff freq
for each pair)

for

$$q'[2i] = q[2i] \cos(m\theta_i) - q[2i+1] \sin(m\theta_i)$$

$$q'[2i+1] = q[2i] \sin(m\theta_i) + q[2i+1] \cos(m\theta_i)$$

d Q · K' depends only on $m - n$

• their absolute position cancel out
on dot product

✗ RoPE is not applied to V as V
carries content, not positional relevance

RoPE & KV cache:

cached k already position m-baked into

∴ never need to reapply RoPE
to cached entries

ISSUE: ✗

cached k's are position-locked

if tokens are evicted; or repositioned
non contiguous; so if a cached k
from representing posn 5 to posn 3

So baked in RoPE rotation is wrong

so it was rotated for posn '5' but pretend to be 3, so need to unrotate

Eg: size 3 ; 6 tokens

cache holds : [3, 4, 5]

positions are : [0, 1, 2]
evicted

K @ posn 3 was RoPE rotated based on posn

$\therefore Q(6) \llcorner (3) \rightarrow \text{relative} : 3$

K at posn 4 (4) : $Q(6) \llcorner (4) : 2$

So

EVICTING POSITIONS [0, 1, 2] affect

any of this $\rightarrow$ NO

but issue after :

Someone applies RoPE after

, $K(3)$ carries rotation for posn - 0

$\therefore$ Evict freely & never re index

Attention Sinks:

very first token in the sequence receives disproportionately high attention from all later ; "The" could be a BOS token

Why?

every row attention weights must sum to 1. Every token MUST distribute

its budget somewhere.

what happens if no strong match with any other

token's key. It has to put extra weight

somewhere; model dumps leftover

attention onto token → garbage collector

←
almost always positioned

why?

→ token that every other token
can attend to even with causal mask

• Sink token's V doesn't need
to carry any info
, just absorb

Soak up extra attn. weights.

so it doesn't get evicted on kv
cache eviction policy that drops lowest
scores. Then what happens?

Attention distribution breaks if we forcefully remove it.

It can't dynamically reassign a new sink.

All the leftover mass, was safely dumped now gets spread across remaining tokens

RANDOMLY not intelligently.

The weighted sum of V now includes random contributions from irrelevant tokens

Per head, Per layer budget:

till now; same compression applied everywhere.

so now when in multi-head attn. where each head focuses on different.

Some heads → Sharp pattern.

↳ one or set of token
get most weight

while others

: flat, diffuse patterns

↳ attn. spread evenly

⊗ head which is sharp → very sensitive to
eviction

so applying same cache budget everywhere
is wasteful.

∴ Adaptive per head budgets allocate
more cache to heads that need it

Per Layer Variation:

Early layers → capture local syntactic
patterns, nearby
relationships.

↳

more local; tolerate compression

Late layers $\rightarrow$ long range semantic relationships

$\hookrightarrow$ more sensitive to losing specific token

So keeping something like

30% cache for layer 2, 70% for layer ₃₀

instead of 50% & 50%